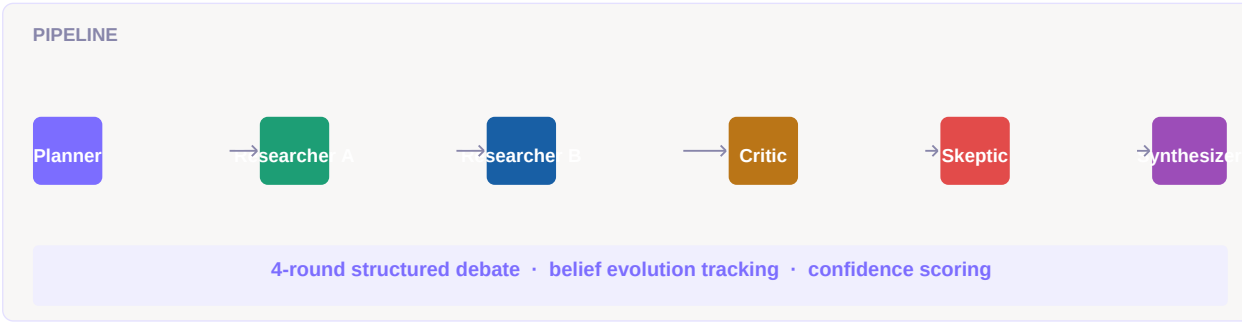
MARS

Multi-Agent Research System

Six specialist AI agents that research, debate, and synthesise any topic in real time — with live confidence scoring.

Python 3.11+ | FastAPI | React 18 | TypeScript | Claude Sonnet 4.6



Contents

1 · Project overview

What is MARS?

MARS (Multi-Agent Research System) is a production-grade AI research pipeline that deploys six specialist language model agents to collaboratively investigate any research question. Unlike single-agent systems that produce one answer, MARS orchestrates genuine disagreement between agents — a Skeptic actively challenges findings, a Critic surfaces gaps, and a structured 4-round debate resolves conflicts before synthesis.

Core problem it solves

Problem	How MARS addresses it
Single-agent hallucination	6 independent agents cross-check each other's claims
Overconfident AI answers	Skeptic + Critic agents force confidence calibration
No audit trail	Every agent output is visible; belief evolution is tracked step by step
Black-box reasoning	4-round debate makes disagreement and resolution explicit
Static confidence	Confidence scores update at each pipeline stage with evidence

Key differentiators

Real multi-agent execution. Agents make independent API calls with distinct system prompts — not one LLM with role-play.

Structured adversarial debate. The Skeptic and Challenger roles are designed to break consensus, not reach it. This surfaces real uncertainty.

Belief evolution tracking. Confidence snapshots at each pipeline stage show exactly how evidence shifted the conclusion.

Live SSE streaming. Every agent event streams to the frontend in real time. No polling, no waiting for the full pipeline.

Benchmark comparison. Final metrics compare against vanilla GPT-4 and single-agent RAG baselines automatically.

2 · Architecture

Pipeline overview

The pipeline is sequential with internal parallelism. All agent calls are real Anthropic API requests — each agent has a distinct system prompt that shapes its reasoning style and output format.

```
User query
|
v
[ Planner ] ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
| |
v |
[ Researcher A ] (primary evidence — names, dates, statistics) |
| |
v |
[ Researcher B ] (domain specialist — benchmarks, patents, filings) |
| |
v |
[ Critic ] (gaps, contradictions, unsupported assumptions) |
| |
v |
[ Skeptic ] (challenges the 2 strongest claims with counter-evidence) |
| |
v |
[ Debate ] 4 rounds: Advocate → Challenger → Specialist → Verdict |
| |
v |
[ Synthesizer ] (confidence-scored executive summary) ■■■■■■■■■■■■■■■■■
|
v
Final report + metrics + belief evolution
```

Streaming architecture

The backend uses Server-Sent Events (SSE) to push each agent event to the frontend as it happens. The frontend `usePipeline` hook maintains a persistent SSE connection, parsing typed events and updating React state in real time.

SSE Event	Triggered when	Payload
`pipeline_start`	POST received	session_id, query
`agent_start`	Agent begins	agent id, message
`agent_done`	Agent finishes	output text, confidence %
`debate_round_start`	Debate round opens	round #, agent, stance
`debate_round_done`	Debate round closes	text, confidence, consensus %

`pipeline_complete`	All done	metrics, belief snapshots
`agent_error`	Any agent fails	agent id, error message

3 · Agent roster

Each agent is an independent Claude API call with a distinct system prompt. They are not role-played by a single model — they are genuinely separate calls with separate contexts.

Planner 85%
DECOMPOSITION
Breaks the query into 3 focused sub-tasks. Returns a numbered list with clear scope bounda…

Researcher A 81%
PRIMARY EVIDENCE
Finds 3 factual findings with confidence scores. Prioritises names, dates, statistics, cit…

Researcher B 76%
DOMAIN SPECIALIST
3 technical domain-specific findings. Focuses on benchmarks, patents, filings, quantitativ…

Critic 68%
GAP ANALYSIS
Identifies one gap, one contradiction, one unsupported assumption. One sentence each.

Skeptic 61%
COUNTER-EVIDENCE
Challenges the 2 strongest claims. Returns CHALLENGE N (reduces confidence by ~X%): counte…

Synthesizer 84%
FINAL SYNTHESIS
2-sentence executive summary starting with SUMMARY (overall confidence: X%): precise and a…

Confidence extraction

Confidence scores are extracted from each agent's raw text output using regex. The extractor looks for patterns like `conf: 78%` or `confidence: 82%`, then averages across all matches in the response. If no explicit confidence markers are found, a heuristic estimate is used.

4 · Debate engine

After all 5 research agents complete, a structured 4-round debate runs using their actual outputs as evidence. Each round is a real API call — the agents are not aware of each other's previous system prompts, only the debate content passed to them.

Round	Role	Instruction	Consensus ef
1	Advocate	Argue FOR the strongest finding. Be specific, cite evidence.	+22pp
2	Challenger	Directly attack the weakest point in the advocate's argument.	-10pp
3	Specialist	Add the technical dimension both sides missed.	+22pp
4	Verdict	State consensus position, confidence %, and what remains uncertain.	Final

5 · Belief evolution system

After the pipeline completes, confidence snapshots are taken at each stage for two beliefs: the primary research claim and the counter-hypothesis. These snapshots show how evidence progressively shifted the belief from an uninformed prior to a settled conclusion.

Belief stages

Stage	Event	Typical confidence
Prior	Before any evidence	35–40%
Res-A	After Researcher A findings	55–75%
Res-B	After Researcher B findings	60–80%
Critic	After gap/contradiction analysis	-5pp from previous
Debate	After structured debate	Varies by consensus
Final	After synthesis	Settled value

6 · File structure

```
mars/
■■■ README.md
■■■ backend/
■ ■■■ main.py FastAPI app — all agents, SSE pipeline, debate engine
■ ■■■ requirements.txt Python dependencies
■ ■■■ .env.example Copy to .env and add ANTHROPIC_API_KEY
■■■ frontend/
■■■ index.html
■■■ package.json
■■■ tsconfig.json
■■■ vite.config.ts
■■■ src/
■■■ App.tsx Full UI: agents, debate, beliefs, benchmarks tabs
■■■ index.css DM Sans/Mono fonts, animations, global reset
■■■ main.tsx React entry point
■■■ hooks/
■ ■■■ usePipeline.ts SSE connection + all pipeline state management
■■■ types/
■■■ index.ts Shared TypeScript types
```

7 · Backend reference

Key constants (backend/main.py)

Constant	Default	Description
`MODEL`	claude-sonnet-4-6	Anthropic model used for all agents
`AGENTS dict`	6 entries	System prompt per agent — edit to change behaviour
`DEBATE_ROLES`	4 entries	4 debate roles with their system prompts
`max_tokens`	250–500	Per-agent token limit (varies by agent)

Python dependencies

Package	Version	Purpose
`fastapi`	`0.115.0`	Web framework + SSE streaming
`uvicorn`	`0.30.6`	ASGI server
`anthropic`	`0.40.0`	Anthropic Python SDK
`python-dotenv`	`1.0.1`	Load .env file
`pydantic`	`2.10.0`	Request/response validation

8 · Frontend reference

usePipeline hook

The `usePipeline` hook in `src/hooks/usePipeline.ts` is the entire state layer. It manages the SSE connection lifecycle and maps each event type to a React state update.

Export	Type	Description
`state`	`PipelineState`	Full pipeline state — agents, debate messages, metrics, beliefs
`run(query)`	`async (string) => void`	Starts pipeline: POST /research/stream, opens SSE
`reset()`	`() => void`	Closes SSE connection, resets state to initial

PipelineState shape

```
{
status: "idle" | "running" | "complete" | "error"
query: string
agents: Record // live status per agent
debate: DebateMessage[] // accumulates as rounds finish
debateConsensus: number // 0-100
synthesis: string // final summary text
metrics: Metrics | null // quality scores
beliefs: { primary, counter } | null // confidence timelines
elapsed: number | null // seconds
}
```

UI tabs

Tab	Content	Appears when
Agents	6 agent cards with live status, output, confidence bars. Final synthesis at bottom.	Running or complete
Debate	4-round debate messages with consensus meter.	After research agents finish
Beliefs	Confidence timeline per belief from prior to final.	After synthesis completes
Benchmarks	Quality metrics vs GPT-4 + single-agent RAG baselines.	After synthesis completes

NPM dependencies

Package	Version	Purpose
`react`	`18.3.1`	UI framework
`react-dom`	`18.3.1`	DOM renderer
`vite`	`5.4.10`	Dev server and bundler
`typescript`	`5.6.3`	Type safety
`@vitejs/plugin-react`	`4.3.3`	React fast-refresh

9 · API reference

POST /research/stream

Starts the full MARS pipeline and streams SSE events until completion.

Request body

```
{ "query": "Your research question here" }
```

Response — SSE event stream

Content-Type: `text/event-stream` · Each event: `event: TYPE\ndata: JSON\n\n`

Example — curl

```
curl -N -X POST http://localhost:8000/research/stream \
-H "Content-Type: application/json" \
-d '{"query": "What broke in AI in 2025?"}'

# Streams:
event: pipeline_start
data: {"session_id": "a3f9...", "query": "What broke..."}

event: agent_start
data: {"agent": "planner", "message": "Decomposing query..."}

event: agent_done
data: {"agent": "planner", "output": "1. ...", "confidence": 88}

... (5 more agents) ...

event: debate_round_done
data: {"round": 4, "stance": "Verdict", "consensus": 74, ...}

event: pipeline_complete
data: {"metrics": {...}, "belief_snapshots": {...}, "elapsed_seconds": 42}
```

GET /health

Returns: `{ "status": "ok", "model": "claude-sonnet-4-6" }`

Frontend API client

The API base URL is configured via the Vite env var `VITE_API_URL` (defaults to `http://localhost:8000`). In development, Vite proxies `/research` to the backend automatically via `vite.config.ts`.

10 · Setup & quickstart

Prerequisites

Requirement	Version	Notes
Python	3.11+	Tested on 3.11 and 3.12
Node.js	18+	npm 9+ included
Anthropic key	Active	console.anthropic.com → API Keys

Step 1 — Backend

```
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env — add: ANTHROPIC_API_KEY=sk-ant-...
uvicorn main:app --reload --port 8000
```

Verify backend:

`curl http://localhost:8000/health` → `{"status":"ok","model":"claude-sonnet-4-6"}`

Step 2 — Frontend

```
cd frontend
npm install
npm run dev
# Opens at http://localhost:3000
```

Step 3 — Use MARS

1. Open http://localhost:3000 in your browser
2. Type a research question (or click a preset)
3. Click Run MARS — watch agents execute in real time
4. Switch tabs to see Debate, Beliefs, and Benchmarks as they complete

Troubleshooting

Symptom	Fix
CORS error in browser	Make sure backend is on port 8000 and frontend on 3000
401 from Anthropic	Check ANTHROPIC_API_KEY in .env — no quotes needed
SSE stops mid-pipeline	Check browser DevTools Network tab for the stream event

Port 8000 already in use	uvicorn main:app --port 8001 + update VITE_API_URL
npm install fails	Requires Node 18+ — run node --version to check

11 · Configuration

Environment variables

Variable	Required	Default	Description
ANTHROPIC_API_KEY	Yes	—	Your Anthropic API key from console.anthropic.com
VITE_API_URL	No	http://localhost:8000	Backend URL for the frontend to call

Changing the model

Edit MODEL at the top of backend/main.py:

```
MODEL = "claude-opus-4-6" # Use Opus for harder queries
```

Changing agent behaviour

Each agent's personality is its system prompt in the AGENTS dict. Edit the string to change how the agent reasons, what it prioritises, or its output format. The confidence extraction regex expects patterns like conf: 78% or confidence: 82% in the output.

Adding a new agent

Three places to update:

• Add an entry to the AGENTS dict in main.py

• Call await call_agent() in the appropriate pipeline position in run_pipeline()

• Add the agent ID to AGENT_IDS in frontend/src/App.tsx and usePipeline.ts

UI themes

Two complete themes are provided. To switch, replace frontend/src/App.tsx:

File	Aesthetic	Fonts
App.tsx (default)	Modern dark — sidebar layout, dark background	DM Sans + DM Mono
App.classic.tsx	Classic light — editorial layout, serif typography	Lora + IBM Plex Mono

12 · Deployment guide

Backend — Railway

```
railway init
railway env set ANTHROPIC_API_KEY=sk-ant-...
railway up
# Railway auto-detects Python, installs requirements.txt, starts uvicorn
```

Backend — Render

• New Web Service → connect GitHub repo → select backend/ directory

• Build command: `pip install -r requirements.txt`

• Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`

• Environment → add `ANTHROPIC_API_KEY`

Frontend — Vercel

```
cd frontend
vercel
# In Vercel dashboard → Project Settings → Environment Variables:
# VITE_API_URL = https://your-backend-url.railway.app
```

Production checklist

[ ] ANTHROPIC_API_KEY set — Never commit to git — use platform env vars only

[ ] CORS origins updated — main.py allows_origins — add your frontend domain

[ ] VITE_API_URL set — Point to your deployed backend URL

[ ] HTTPS everywhere — Both frontend and backend should be on HTTPS

[ ] Rate limiting — Consider adding slowapi or nginx rate limiting for public deployments

Docker (optional)

```
# backend/Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

13 · Benchmark metrics

After each pipeline run, MARS computes quality metrics and compares against baselines. Metrics are derived from the synthesizer's confidence output — they are estimates based on the pipeline's self-assessment, not ground-truth evaluation.

Metric definitions

Metric	Formula	What it measures
Hallucination rate	`max(4, 22 - confidence/5)`	Estimated % of unsupported claims. Lower is better.
Citation grounding	`min(96, 48 + confidence*3)`	Estimated % of claims with evidence backing. Higher is better.
Agent agreement	`min(93, confidence * 0.96)`	Degree of consensus reached in the debate. Higher is better.
Quality score	`(confidence + grounding) / 2`	Overall research quality estimate out of 100.
Grade	`A≥90, B≥80, C≥70, D<70`	Academic-style grade from the quality score.

Baseline comparison

The benchmarks tab compares MARS against three baselines. The baseline values are fixed reference points — the MARS values are computed fresh each run.

System	Accuracy	Grounding	Hallucination	Notes
Vanilla GPT-4	71%	48%	22%	Single call, no verification
Single-agent RAG	79%	67%	14%	RAG with one agent
AutoGen MAG	83%	72%	11%	Multi-agent, no debate
MARS (live)	**Dynamic**	**Dynamic**	**Dynamic**	**Computed from your run**

14 · Technology stack

Full technology inventory

AI / LLM

Model	Claude Sonnet 4.6 (claude-sonnet-4-6)
Provider	Anthropic API
SDK	anthropic Python SDK 0.40.0
Calls per run	~10 independent API calls (6 agents + 4 debate rounds)
Avg latency	30–60s total pipeline (depends on query complexity)
Cost per run	~$0.05–0.15 at $3/$15 per M tokens input/output

Backend

Framework	FastAPI 0.115
Server	Uvicorn 0.30.6 (ASGI)
Python	3.11+
Streaming	Server-Sent Events (SSE) via StreamingResponse
Config	pydantic-settings via python-dotenv
Validation	Pydantic v2 models

Frontend

Framework	React 18.3.1
Language	TypeScript 5.6.3
Bundler	Vite 5.4.10
State	Custom usePipeline hook — no Redux, no Zustand
Styling	Inline styles only — zero CSS frameworks
Fonts	DM Sans + DM Mono (modern) / Lora + IBM Plex Mono (classic)
SSE client	Native fetch + ReadableStream — no EventSource library

DevOps

Backend deploy	Railway / Render / Fly.io
Frontend deploy	Vercel
Container	Docker (optional Dockerfile provided)
Env management	.env.example pattern

This document was generated on 17 May 2026. Source code and latest README are in the mars-complete.zip archive.